Parallel Computer Architecture & Programming
Problem Statement 5

Scalable Parallel K-Means Clustering for Big Data Using MPI and CUDA

Mini-Project Synopsis

`colab` branch implementation

At large point counts, the bottleneck in K-means clustering is the repeated nearest-centroid scan. This project distributes that work across MPI ranks and offloads the per-rank assignment step to a CUDA kernel. Three binaries serial, MPI-only, and hybrid MPI+CUDA share one mathematical core and one data format, so correctness comparisons are well-defined.

Field	Detail
Course	Parallel Computer Architecture & Programming (PS-5)
Problem statement	Scalable parallel clustering for large point sets
Project title	Scalable Parallel K-Means Clustering for Big Data Using MPI and CUDA
Repository branch	`colab`

Team

1. Tirth Miteshkumar Patel Reg. 240905272 Roll 27 CSE D
Distributed-memory MPI design, dataset partitioning, synchronization, scaling analysis, and the shared numerical core.

2. Aarush Arora Reg. 240905266 Roll 26 CSE D
CUDA assignment kernel, device-side reduction, CPU fallback when no GPU is present.

3. Udit Asthana Reg. 240905310 Roll 31 CSE D
Integration of the MPI driver and the CUDA kernel into `kmeans_hybrid`: the shared assignment contract, the `USE_CUDA_KERNEL` build, and the runners that validate the bound binary against the serial reference.

Contents

1 Abstract **2**

2 Introduction **3**

3 Problem Statement **3**

4 Algorithms **4**
4.1 Lloyd Iteration 4
4.2 Initialization 4

5 System Design **5**
5.1 MPI Layer 5
5.2 CUDA Kernel 5
5.3 Integration Contract 5
5.4 Shared Header 6

6 Timing and Communication **6**

7 Datasets and Benchmark Plan **6**

8 Performance Evaluation **7**
8.1 Strong Scaling 7
8.2 Weak Scaling 7

9 Correctness and Validation **7**

10 Expected Outcomes **8**
10.1 Known Limits 8

11 Work Allocation **8**

12 Conclusion **9**

1. Abstract

K-means clusters N points in D dimensions into K groups by alternating nearest-centroid assignment with centroid update. One full assignment pass costs $O(NKD)$, which is the step that makes a single-threaded implementation slow at scale. This project parallelizes that step at two levels.

At the distributed level, points are split across MPI ranks. Each rank assigns its own block and contributes partial cluster sums to a global `MPI_Allreduce`; the communication volume per iteration is $\Theta(KD)$, independent of N. At the intra-node level, the CUDA kernel gives one thread to each local point. A CPU fallback activates when no usable device is found.

Three binaries share `include/kmeans_common.h`, which defines the data layout, the distance, the seeding, and the centroid update. A Python checker compares runs by

relative SSE and by minimum-cost centroid matching. A scaling harness records strong- and weak-scaling rows to a CSV.

2. Introduction

Lloyd's batch form of K-means [1, 2] runs two operations per iteration. First, every point is assigned to the centroid minimizing squared Euclidean distance. Second, each centroid is replaced by the mean of the points assigned to it. The assignment step reads every coordinate of every point against every centroid, giving a cost of $O(NKD)$ per iteration. The update step reduces $K \times D$ sums, which is small.

The assignment of one point does not depend on any other point's assignment in the same iteration. Points are therefore split across processes, and each process handles its own block. The global centroid update requires only a sum of partial sums and a sum of partial counts a single all-reduce per iteration. The communication volume stays at $\Theta(KD)$ regardless of N, which is the basis for expected near-linear speedup on large datasets, as established by Dhillon and Modha [4].

Within one rank, the distance loop over local points is wide and uniform the right shape for GPU parallelism. The CUDA kernel assigns one thread per local point [5]. Current work uses atomic updates for the partial sums [6]. The key design constraint is that the GPU kernel must honor the same inputs and outputs as the CPU assignment function, so the MPI iteration does not acquire a second control path.

3. Problem Statement

Problem Statement 5 asks for a scalable parallel clustering program. The required deliverables have three parts: a serial reference, a distributed-memory MPI implementation, and a GPU implementation of the assignment step embedded in the same distributed iteration.

The engineering constraints that follow are concrete.

- All three binaries must read the same bytes from disk and apply the same update rule, so that a mismatch in the objective is not ambiguous.
- Only one rank reads the full matrix. After the scatter, each rank holds only its own contiguous block.
- Communication time and computation time are recorded separately per iteration, with per-rank extremes retained.
- The GPU entry point satisfies the same function signature as the CPU assignment function. A missing device falls back to the CPU path without changing the MPI control flow.

The stopping rule used throughout is maximum centroid shift: the iteration loop ends when

$$\Delta = \max_{k:\, n_k > 0} \left\| \mu_k^{\text{new}} - \mu_k^{\text{old}} \right\|_2 < \tau,$$

where $\tau = 10^{-4}$ by default and the iteration cap is 100.

4. Algorithms

4.1. Lloyd Iteration

Let $x_i \in \mathbb{R}^D$ for $i = 1, \ldots, N$, with current centroids $\mu_1, \ldots, \mu_K$. The assignment of point i is

$$c(i) = \arg\min_{k \in \{1,\ldots,K\}} \left\| x_i - \mu_k \right\|_2^2. \tag{1}$$

The objective is the within-cluster sum of squared errors,

$$\mathrm{SSE} = \sum_{i=1}^{N} \left\| x_i - \mu_{c(i)} \right\|_2^2. \tag{2}$$

After assignment, each non-empty centroid is updated as

$$\mu_k \leftarrow \frac{1}{|C_k|} \sum_{x \in C_k} x. \tag{3}$$

In parallel, the numerator and denominator of (3) are assembled from per-rank partial sums $S_k^{(r)}$ and counts $n_k^{(r)}$:

$$S_k = \sum_{r=0}^{P-1} S_k^{(r)}, \quad n_k = \sum_{r=0}^{P-1} n_k^{(r)}, \quad \mu_k = S_k / n_k. \tag{4}$$

Accumulators are double precision; stored points and centroids are single precision. Squared distance is used throughout, which avoids a square root and preserves the arg-minimum.

4.2. Initialization

Two seeded methods are available. Uniform sampling shuffles the point-index vector with `std::mt19937` at the given seed and copies the first K rows into the centroid table.

k-means++ [3] draws the first center uniformly from $\{0, \ldots, N-1\}$. Each subsequent center is drawn with probability proportional to the squared distance from the nearest already-chosen center,

$$\Pr(x) = \frac{D(x)^2}{\sum_{x'} D(x')^2}. \tag{5}$$

Both methods run on rank 0. The resulting centroid table is broadcast before the first assignment.

5. System Design

Binary	Build target	Assignment
`bin/kmeans_serial`	`make serial`	`assign_points_cpu` on all N points
`bin/kmeans_mpi`	`make mpi`	`assign_points_cpu` on the local block
`bin/kmeans_hybrid`	`make hybrid`	`launch_assignment_kernel` on the local block

5.1. MPI Layer

`src/kmeans_mpi.cpp` manages the distributed iteration. Rank 0 reads the matrix, broadcasts N and D, and draws the initial centroids. Points are partitioned with

$$n_r = \left\lfloor N/P \right\rfloor + \mathbf{1}[r < N \bmod P], \tag{6}$$

then scattered with `MPI_Scatterv`. After initialization, rank 0 releases the full matrix. Each iteration zeros local sum and count buffers, calls `do_assignment_step`, and all-reduces three arrays: $K \times D$ double-precision coordinate sums, K integer counts, and one double-precision partial SSE. Every rank then calls `update_centroids` on the reduced globals. Labels are gathered at rank 0 with `MPI_Gatherv` after the loop.

`MPI_Allreduce` is necessary at each iteration because every rank needs the updated centroids before the next assignment. A reduce-then-broadcast would add a second collective; dividing the summed sums by the summed counts locally on each rank avoids that.

5.2. CUDA Kernel

`src/cuda_assignment_kernel.cu` implements (1) on the device. The kernel uses a one-dimensional grid of 256-thread blocks; each thread handles one local point. The thread scans the K centroids, selects the minimizing index, and updates cluster sums and counts with `atomicAdd`. On pre-6.0 architectures, double-precision atomic addition is replaced with a compare-and-swap loop on the 64-bit representation.

The host launcher `launch_assignment_kernel` allocates device buffers, copies points and centroids, launches the kernel, synchronizes, copies results back, and frees device memory all on every Lloyd iteration. Two failure paths exist: absent device count and failed allocation. Both redirect to `assign_points_cpu`. The fallback fires once and prints a warning; subsequent iterations use the same CPU path.

5.3. Integration Contract

`do_assignment_step` in the MPI driver is the only compile-time branch on `USE_CUDA_KERNEL`. Both functions receive the local points, the local length, D, the centroid table, K, a label buffer, a double-precision sum buffer, and an integer count buffer; both return the local SSE.

The hybrid Makefile rule compiles `src/cuda_assignment_kernel.cu` with `nvcc` and links it into the MPI driver with `-DUSE_CUDA_KERNEL` and `libcudart`. `make all` builds

only the serial and MPI binaries. A machine without `nvcc` still produces the full CPU path.

5.4. Shared Header

`include/kmeans_common.h` is the single source of the binary format, the partition rule, both initializers, the CPU assignment, the centroid update, and the output writers. All functions are `inline`. The on-disk format places a 12-byte header (magic `0x4B4D4E53`, N, D as native 32-bit integers) before $N \times D$ row-major `float32` values. Contiguity is required by both `MPI_Scatterv` and `cudaMemcpy`.

6. Timing and Communication

Two timestamps bracket `do_assignment_step` inside each iteration; the outer interval is the computation time. A third timestamp follows the three all-reduces; the intervening interval is the communication time. After the loop, each per-rank total is reduced with `MPI_MIN`, `MPI_MAX`, and `MPI_SUM`. The metrics file stores the average as the sum divided by the rank count, and the minimum and maximum alongside.

The communication-overhead fraction reported per run is

$$f_{\text{comm}} = \frac{\overline{T}_{\text{comm}}}{T_{\text{total}}}, \tag{7}$$

where $\overline{T}_{\text{comm}}$ is the average per-rank communication time and T_{total} is the barrier-to-barrier wall time of the iteration loop.

On the hybrid path, the computation interval includes device allocation, host-to-device copy, kernel execution, synchronization, and device-to-host copy. It is not a pure arithmetic time. A hybrid-versus-MPI comparison should state that the compute field contains data movement until a profiling pass separates those components.

7. Datasets and Benchmark Plan

Dataset	**Shape**	**Role**
Synthetic	$N \times D$, configurable	Scaling, correctness, Colab demo
MNIST [8]	70,000 × 784	High-dimensional public matrix; $K = 10$ matches digit count
KDD Cup 1999 (10%) [9]	494,021 × ∼118 after one-hot encoding	Mixed numeric/categorical traffic matrix
US Census 1990 [10]	2,458,285 × ∼68	Large-N throughput workload

Every file that reaches a solver has been produced by either `scripts/generate_dataset.py` or `scripts/prepare_real_data.py`; the solver never parses CSV. Columns are shifted

to zero mean and scaled to unit variance by default, because squared Euclidean distance is sensitive to coordinate scale. The Colab notebook and default runners use a synthetic $10^5 \times 16$ matrix with 10 generating clusters, which is large enough to exercise every code path and small enough for an interactive session.

8. Performance Evaluation

8.1. Strong Scaling

Strong scaling fixes the dataset and varies the rank count over $\{1, 2, 4, 8, 16\}$. Speedup and efficiency at rank count P are

$$S(P) = \frac{T(1)}{T(P)}, \quad E(P) = \frac{S(P)}{P}, \tag{8}$$

where $T(P)$ is `total_time_s`. When iteration counts differ across rows, $T(P)$/iterations is the comparable figure; the SSE column confirms that rows sharing a seed followed the same centroid trajectory.

$S(P)$ is expected to rise while $(N/P)KD$ dominates the $\Theta(KD)$ all-reduce. It flattens as per-rank local work shrinks relative to communication and, on the hybrid path, GPU launch and transfer overhead. The scaling harness is built to show that flattening, including at rank counts that oversubscribe a small VM.

8.2. Weak Scaling

Weak scaling holds the work per rank approximately constant. Datasets `<prefix>_r`P`.bin` are generated with N proportional to P and the same D. Ideal behavior keeps $T(P)$ near $T(1)$. A rise in `total_time_s` reflects the growth of the all-reduce with P, or GPU contention when the hybrid binary is under test.

9. Correctness and Validation

K-means finds a local minimum of (2). Two runs from the same initialization at the same seed must agree within rounding, but cluster labels are an arbitrary numbering and atomic GPU reductions are not bit-identical to a serial loop.

Given a reference prefix and a candidate prefix, `scripts/validate_correctness.py` applies four tests.

1. The metrics files must agree on N, D, and K.
2. The relative SSE gap must fall within $\varepsilon = 10^{-3}$:

$$\frac{|\mathrm{SSE}_a - \mathrm{SSE}_b|}{\max(|\mathrm{SSE}_a|,\, 10^{-12})} \leq \varepsilon. \tag{9}$$

3. Centroids are paired by a minimum-cost assignment on the matrix of pairwise Euclidean distances. SciPy solves this with the Jonker–Volgenant algorithm [12]. Each paired distance must be at most $\delta = 10^{-2}$.

4. When both assignment files exist, every label must lie in $\{0, \ldots, K-1\}$ and each file must contain exactly N labels.

Each binary also checks internally that the sum of cluster counts equals N before writing output. A mismatch is printed on stderr.

10. Expected Outcomes

The deliverables present on the `colab` branch are as follows.

- A serial reference that writes SSE, a centroid table, and a full label vector.
- An MPI implementation producing the same output from multiple ranks, with per-iteration computation and communication times and cross-rank extremes.
- A hybrid implementation that calls the CUDA kernel when a device is available and the CPU loop otherwise, without any change to the MPI iteration.
- A binary dataset format with two writers: one synthetic, one for MNIST, KDD Cup 1999, and US Census 1990.
- A checker with explicit tolerances and a scaling harness that appends strong- and weak-scaling rows to a CSV.

The quantitative results from a particular machine are a CSV and a set of metrics files, interpreted with (8) and (7). An expected qualitative pattern is a communication fraction that falls as N grows at fixed K and D, and a strong-scaling curve that rises while local assignment dominates the all-reduce time.

10.1. Known Limits

- Rank 0 holds the full matrix through initialization. Both the matrix read and the k-means++ scan are serial and require $O(ND)$ memory on that rank.
- The hybrid launcher allocates and frees device buffers on every iteration. All ranks use the default CUDA device, so multiple ranks on one GPU serialize through the driver rather than running in parallel.
- The kernel uses global-memory atomics without shared-memory or register blocking, which matters most at high D [6].
- Empty clusters are not reseeded.

11. Work Allocation

The team has three workstreams with a clear boundary at `do_assignment_step`.

Tirth Miteshkumar Patel owns the distributed-memory solver and the shared core: `include/kmeans_common.h`, `src/kmeans_serial.cpp`, and `src/kmeans_mpi.cpp`. This covers rank management, `plan_partition`, the scatter, centroid broadcast, the all-reduce of sums, counts, and SSE, local centroid reconstruction, the gather of labels, and the scaling and checker scripts.

Aarush Arora owns the GPU assignment workstream: `src/cuda_assignment_kernel.cu`, the device kernel, double-precision atomics including the pre-6.0 compare-and-swap path, the host launcher, and the runtime CPU fallback. A kernel change is accepted when the checker passes on matched centroids and SSE within the tolerances of Section 9.

Udit Asthana integrates the two kernels into one hybrid executable. The work has three parts: matching the `extern "C"` signature of `launch_assignment_kernel` to `do_assignment_step`; writing the `hybrid` Makefile rule; and providing `run_project.sh`, `run_colab.sh`, and the Colab notebook, which build all three binaries, run them on the same matrix with the same seed and initialization, and accept the hybrid run when the checker passes.

12. Conclusion

The project implements K-means as a data-parallel Lloyd iteration. One header defines the data format, the distance, the seeding, and the centroid update. The serial driver runs that iteration on one core and records the reference SSE and centroids. The MPI driver partitions the points, assigns locally, all-reduces the partial sums and counts, and rebuilds centroids on every rank. The CUDA kernel replaces the local assignment with one thread per point and atomic cluster statistics, falling back to the shared CPU loop when the device is unavailable. Separate timers isolate computation from communication, and a checker compares runs by relative SSE and minimum-cost centroid matching.

Tirth Miteshkumar Patel provides the MPI distribution and the global update. Aarush Arora provides the CUDA distance work. Udit Asthana binds the two so the hybrid binary runs as one solver.

References

[1] S. P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[2] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.

[3] D. Arthur and S. Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007.

[4] I. S. Dhillon and D. S. Modha. A data-clustering algorithm on distributed memory multiprocessors. In *Large-Scale Parallel Data Mining*, Lecture Notes in Computer Science, volume 1759, pages 245–260. Springer, 2000.

[5] R. Farivar, D. Rebolledo, E. Chan, and R. Campbell. A parallel implementation of K-means clustering on GPUs. In *Proceedings of the International Conference on*

Parallel and Distributed Processing Techniques and Applications (PDPTA), pages 340–345, 2008.

[6] Y. Li, K. Zhao, X. Chu, and J. Liu. Speeding up k-means algorithm by GPUs. *Journal of Computer and System Sciences*, 79(2):216–229, 2013.

[7] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023.

[8] Y. LeCun, C. Cortes, and C. J. C. Burges. The MNIST database of handwritten digits. `http://yann.lecun.com/exdb/mnist/`.

[9] S. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. Chan. KDD Cup 1999 Data. UCI Machine Learning Repository, 1999.

[10] C. Meek, B. Thiesson, and D. Heckerman. US Census Data (1990). UCI Machine Learning Repository, 2001.

[11] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[12] H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.